RelayOps — Round 2 Build Report

Evidence-backed employee handover · Local Round 2 implementation

Project summary

RelayOps turns authorised project records into an evidence-backed employee handover. Project records are imported, privacy-filtered, stored as source passages, reconstructed into findings, and tied to exact source quotations. Contradictions and missing evidence stay visible. A human reviewer makes the decision; a project lead approves a versioned snapshot before an incoming contributor can see it.

The ready-to-use Project Chronos workspace models the Round 1 fintech migration scenario with 33 synthetic records: six architecture/runbook sources, eighteen GitHub PR/issue records, a Linear task record, four Slack channel records including restricted evidence, and the original cross-source conflict example. The local deterministic workflow needs no user API credentials. Gemini extraction and private-repository import require a configured key/token.

Completed features

- Public product homepage and a working sign-in screen.
- Local Supabase Auth sample accounts for project lead and incoming contributor.
- Five-stage workflow: Connect, Reconstruct, Verify, Review, and Handover.
- Project Chronos synthetic workspace persisted in PostgreSQL, with the six-document, eighteen-repository-record, Linear, and four-channel Slack corpus described in Round 1.
- Import Markdown (including Notion page exports), RelayOps JSON, Linear issue JSON, Slack channel/thread JSON, and DiscordChatExporter channel/thread JSON with structural validation, a 50-record import ceiling, bounded file size/count, thread grouping, timestamp/author provenance, clearance preservation, and duplicate checks.
- Flag a GitHub contributor who authored more than 70% of imported commits in the last 14 days when linked docs, tasks, and discussions do not mention them; the finding is marked as missing evidence and cites the contributing commit records.
- Import a public GitHub README, root/docs Markdown files, recent commits and selected commit/PR diffs, issues/comments, pull requests/reviews/conversation comments/code review comments; optional private-repository token is request-only. Pagination and the returned batch are bounded (50 records maximum).
- Redact common API credentials, bearer tokens, private keys, emails, and phone numbers before storage, using Presidio when available and deterministic scanner fallback otherwise.
- Store sources, passages, claims, citations, graph entities/edges, review decisions, approval snapshots, and receipt acknowledgements in PostgreSQL with pgvector support.
- Reconstruct supported facts deterministically; optional Gemini/LangGraph extraction can produce structured facts, embeddings, and graph edges when a valid key/quota is configured.
- Validate every persisted citation against the exact stored passage and quoted text. Model output cannot approve or resolve a disagreement.
- Show disagreements and evidence gaps explicitly in Verify.
- Search embedded source passages by natural-language query with pgvector cosine similarity; the search RPC runs as the signed-in user so project membership and clearance RLS filter matches before content is returned.
- Human review actions use optimistic claim versions; any changed claim needs a new decision. A heartbeat-backed project lock prevents source import or extraction from racing with review and approval.
- A lead approval gate blocks incomplete handovers. Versioned snapshots are immutable and repeated approval of one revision returns its existing snapshot.
- Contributors receive only an approved snapshot and only data at their clearance level; source text, citations, and related evidence are filtered server-side and protected by RLS.
- Incoming contributor can acknowledge the specific approved snapshot.

Technology and implementation details

- **Web application:** Next.js 16.4.5 App Router, React 19.3.1, TypeScript 5.9.3. Server-rendered routes protect the workspace; server route handlers validate inputs with Zod and use verified Supabase sessions.
- **Database and identity:** local Supabase stack, PostgreSQL 17, Supabase Auth, pgvector, SQL migrations, seeded synthetic users and data. RLS enforces project membership and clearance; server APIs apply matching checks.
- **Python extractor:** FastAPI and Pydantic service with LangGraph orchestration. Presidio and deterministic scanners redact content before model use. Gemini produces structured extraction and embeddings when configured.
- **GitHub ingestion:** Octokit makes paginated, bounded requests for README/docs, commits and selected file diffs, issues and comments, pull requests and changed-file diffs, reviews, conversation comments, and code review comments. Tokens are used for the request and are not persisted.
- **Orphaned ownership heuristic:** a deterministic 14-day author-share calculation emits an evidence-gap finding above the Round 1 70% threshold only when no imported document, task, or discussion mentions that contributor; the claim cites the author's actual imported commit messages for human investigation.
- **Evidence integrity:** passages are persisted before claims. Citation validation checks passage membership, exact quote containment, and Unicode code-point offsets consistently across Python, TypeScript, and PostgreSQL. Query embeddings and 768-dimensional passage embeddings are compared by a bounded pgvector cosine-similarity RPC. The RPC uses `SECURITY INVOKER`, and its results remain constrained by project/source/passage RLS even if a caller requests a higher clearance. Citationless imported findings become missing evidence. Version checks prevent stale decisions; heartbeat-backed mutation locks and database write-fence triggers serialize ingestion/extraction with review/approval and reject stale mutation tokens. Approval writes an immutable versioned snapshot that is the contributor-facing source of truth.
- **Privacy boundary:** imported content is treated as untrusted data, model prompts prohibit following source instructions, and generated findings require locally verified citations. Models never make approval decisions.

Main data flow

Notion Markdown / RelayOps, Linear, Slack, Discord JSON / GitHub records

```
→ validate and redact

→ persist sources and exact passages

→ deterministic reconstruction or optional Gemini/LangGraph extraction

→ verify citations and preserve conflicts

→ reviewer decisions and lead approval

→ versioned approved snapshot

→ clearance-filtered incoming view and receipt acknowledgement
```

Changes from the Round 1 proposal

The initial Round 2 build temporarily replaced the planned database, server-side access, GitHub ingestion, vector/graph storage, and LLM extractor with browser-local deterministic logic. That did not meet the Round 1 plan. This build restores the planned architecture: Supabase/PostgreSQL and RLS, server-backed persistence, GitHub imports, a privacy-filtering FastAPI/LangGraph Gemini extractor, vector/temporal graph data, and human approval. A deterministic path remains as a no-key baseline for judges; it is not presented as a substitute for the planned AI path. General-purpose workspace creation and team invitations are not implemented in Round 2.

Known limitations / what does not work yet

- A live Gemini extraction request was attempted with the configured key but the extractor returned HTTP 502. The key is not verified for extraction; the provider root cause is unknown and no repeated request was made. Semantic retrieval requires successful embeddings and is not verified end to end.
- Private GitHub repository import requires a read-only fine-grained token. The import is bounded to 50 recent records, selected diffs, and root/docs Markdown; it does not mirror the full repository or watch for later changes.
- The public link is not deployed: static GitHub Pages cannot run these APIs. Render and hosted Supabase still need production configuration.
- Only the seeded Project Chronos workspace and local demo accounts are supplied. Arbitrary workspaces and invitations are not included.
- Presidio requires the extractor service; when it is unavailable, the API reports that it is using the deterministic Node redaction fallback.
- An interrupted source/extraction request keeps approval blocked until its 15-minute mutation lease expires; retry the source import or extraction to recover and reconcile the project before reviewing.
- Deployment is not verified. Local source, review, approval, and receipt behavior is covered by the relevant tests and application checks.

Three planned Round 3 improvements

1. **Voice exit micro-interview:** collect a departing teammate's answer to a specific evidence gap, timestamp and attach the response as a source, then require human review before it enters an approved handover.
2. **Dependency graph and Day-1 runbook:** visualize blocker/task dependencies and turn approved dependencies and next actions into an interactive first-week checklist for the incoming teammate.
3. **Tamper-evident audit, version diff, and receipt export:** record source-to-decision changes in a verifiable event history, tie recipient sign-off to the approved snapshot, and export the handover as a PDF.

Local judge walkthrough

1. Start Docker Desktop, run `npx supabase start`, then `npx supabase db reset`.
2. Run `./scripts/setup-local-env.ps1` once to write local-only settings to `.env.local`.
3. Start the extractor in one terminal with `./services/extractor/scripts/start.ps1` and the web app in another with `npm run dev`.
4. Open <http://localhost:3000>, choose **Open workspace**, and sign in as `lead@relayops.local` with password `RelayOpsDemo!2026`.
5. Connect/import records, inspect Reconstruct and Verify, resolve or accept evidence statuses in Review, and approve the project.
6. Sign out; sign in as `contributor@relayops.local` with the same local demo password to see the clearance-filtered approved snapshot and acknowledge receipt.
7. Optional: configure `GEMINI_API_KEY` in ignored `.env.local`, restart both services, and run Gemini extraction from Reconstruct. Optional: import a public GitHub repository without a token.

Submission links and current status

- Working local app: <http://localhost:3000> (available only while local services are running).
- Source repository: <https://github.com/karthik-258-code/RelayOps/tree/codex/relayops-round2>.
- Public judge link: not deployed yet. Do not submit the former static GitHub Pages URL as the Round 2 application; it cannot execute this server-backed build.